

ELEC 576 / COMP 576 – Assignment 0

Anqi Li

September 9, 2026

Task 1

Output of `conda info`:

```
active environment : elec576
active env location : /rhf-wq8/al318/miniconda3/envs/elec576
shell level : 1
user config file : /home/al318/.condarc
populated config files : /rhf-wq8/al318/miniconda3/.condarc
                        /rhf-wq8/al318/miniconda3/condarc.d/anaconda-auth.yml
conda version : 26.3.2
conda-build version : not installed
python version : 3.13.13.final.0
solver : libmamba (default)
virtual packages : __archspec=1=zen5
                  __conda=26.3.2=0
                  __cuda=13.2=0
                  __glibc=2.35=0
                  __linux=5.15.0=0
                  __unix=0=0
base environment : /rhf-wq8/al318/miniconda3 (writable)
conda av data dir : /rhf-wq8/al318/miniconda3/etc/conda
conda av metadata url : None
channel URLs : https://repo.anaconda.com/pkgs/main/linux-64
               https://repo.anaconda.com/pkgs/main/noarch
               https://repo.anaconda.com/pkgs/r/linux-64
               https://repo.anaconda.com/pkgs/r/noarch
package cache : /rhf-wq8/al318/miniconda3/pkgs
               /home/al318/.conda/pkgs
envs directories : /rhf-wq8/al318/miniconda3/envs
               /home/al318/.conda/envs
temporary directory : /tmp
platform : linux-64
user-agent : conda/26.3.2 requests/2.33.1 CPython/3.13.13 Linux/5.15.0-190-generic ubuntu/22.04.5
glibc/2.35 solver/libmamba conda-libmamba-solver/26.4.0 libmambapy/2.3.2 aau/0.7.6 c/. s/. e/.
UID:GID : 100228:100
netrc file : None
offline mode : False
```

Task 2

I ran the commands in the “Linear Algebra Equivalents” table in IPython. Most rows use a 3×3 matrix `a`. The slicing rows need more rows and columns, so those use a 21×9 matrix `m`. Two commands from the table did not run as written on the current SciPy: `signal.resample` needs an integer number of samples, and the `np.ix_(np.r_[0:9:], np.r_[0:6:])` line is missing a closing parenthesis. I fixed both.

```

In [1]: import numpy as np

In [2]: import scipy.linalg

In [3]: from scipy import signal

In [4]: from scipy.sparse.linalg import eigs, cg

In [5]: np.set_printoptions(precision=4, suppress=True, linewidth=100)

In [6]: # --- matrices used throughout ---

In [7]: a = np.array([[1., 2., 3.], [4., 5., 6.], [7., 8., 10.]])

In [8]: a
Out[8]:
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.],
       [ 7.,  8., 10.]])

In [9]: m = np.arange(1., 190.).reshape(21, 9)

In [10]: m
Out[10]:
array([[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.],
       [10., 11., 12., 13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24., 25., 26., 27.],
       [28., 29., 30., 31., 32., 33., 34., 35., 36.],
       [37., 38., 39., 40., 41., 42., 43., 44., 45.],
       [46., 47., 48., 49., 50., 51., 52., 53., 54.],
       [55., 56., 57., 58., 59., 60., 61., 62., 63.],
       [64., 65., 66., 67., 68., 69., 70., 71., 72.],
       [73., 74., 75., 76., 77., 78., 79., 80., 81.],
       [82., 83., 84., 85., 86., 87., 88., 89., 90.],
       [91., 92., 93., 94., 95., 96., 97., 98., 99.],
       [100., 101., 102., 103., 104., 105., 106., 107., 108.],
       [109., 110., 111., 112., 113., 114., 115., 116., 117.],
       [118., 119., 120., 121., 122., 123., 124., 125., 126.],
       [127., 128., 129., 130., 131., 132., 133., 134., 135.],
       [136., 137., 138., 139., 140., 141., 142., 143., 144.],
       [145., 146., 147., 148., 149., 150., 151., 152., 153.],
       [154., 155., 156., 157., 158., 159., 160., 161., 162.],
       [163., 164., 165., 166., 167., 168., 169., 170., 171.],
       [172., 173., 174., 175., 176., 177., 178., 179., 180.],
       [181., 182., 183., 184., 185., 186., 187., 188., 189.]])

In [11]: v = np.array([0.1, 0.6, 0.9])

In [12]: u = np.array([[0.2, 0.8, 0.5], [0.6, 0.4, 0.9]])

In [13]: # --- ndims / numel / size ---

In [14]: np.ndim(a)
Out[14]: 2

In [15]: a.ndim
Out[15]: 2

In [16]: np.size(a)
Out[16]: 9

In [17]: a.size
Out[17]: 9

In [18]: np.shape(a)
Out[18]: (3, 3)

```

```

In [19]: a.shape
Out[19]: (3, 3)

In [20]: a.shape[1]
Out[20]: 3

In [21]: # --- construction ---

In [22]: np.array([[1., 2., 3.], [4., 5., 6.]])
Out[22]:
array([[1., 2., 3.],
       [4., 5., 6.]])

In [23]: p = np.array([[1., 2.], [3., 4.]]); q = np.array([[5., 6.], [7., 8.]])

In [24]: np.block([[p, q], [q, p]])
Out[24]:
array([[1., 2., 5., 6.],
       [3., 4., 7., 8.],
       [5., 6., 1., 2.],
       [7., 8., 3., 4.]])

In [25]: # --- indexing and slicing ---

In [26]: v[-1]
Out[26]: np.float64(0.9)

In [27]: m[1, 4]
Out[27]: np.float64(14.0)

In [28]: m[1]
Out[28]: array([10., 11., 12., 13., 14., 15., 16., 17., 18.])

In [29]: m[1, :]
Out[29]: array([10., 11., 12., 13., 14., 15., 16., 17., 18.])

In [30]: m[0:5]
Out[30]:
array([[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.],
       [10., 11., 12., 13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24., 25., 26., 27.],
       [28., 29., 30., 31., 32., 33., 34., 35., 36.],
       [37., 38., 39., 40., 41., 42., 43., 44., 45.]])

In [31]: m[:5]
Out[31]:
array([[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.],
       [10., 11., 12., 13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24., 25., 26., 27.],
       [28., 29., 30., 31., 32., 33., 34., 35., 36.],
       [37., 38., 39., 40., 41., 42., 43., 44., 45.]])

In [32]: m[0:5, :]
Out[32]:
array([[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.],
       [10., 11., 12., 13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24., 25., 26., 27.],
       [28., 29., 30., 31., 32., 33., 34., 35., 36.],
       [37., 38., 39., 40., 41., 42., 43., 44., 45.]])

In [33]: m[-5:]
Out[33]:
array([[145., 146., 147., 148., 149., 150., 151., 152., 153.],
       [154., 155., 156., 157., 158., 159., 160., 161., 162.],
       [163., 164., 165., 166., 167., 168., 169., 170., 171.],
       [172., 173., 174., 175., 176., 177., 178., 179., 180.],
       [181., 182., 183., 184., 185., 186., 187., 188., 189.]])

```

```

In [34]: m[0:3, 4:9]
Out[34]:
array([[ 5.,  6.,  7.,  8.,  9.],
       [14., 15., 16., 17., 18.],
       [23., 24., 25., 26., 27.]])

In [35]: m[np.ix_([1, 3, 4], [0, 2])]
Out[35]:
array([[10., 12.],
       [28., 30.],
       [37., 39.]])

In [36]: m[2:21:2, :]
Out[36]:
array([[ 19.,  20.,  21.,  22.,  23.,  24.,  25.,  26.,  27.],
       [ 37.,  38.,  39.,  40.,  41.,  42.,  43.,  44.,  45.],
       [ 55.,  56.,  57.,  58.,  59.,  60.,  61.,  62.,  63.],
       [ 73.,  74.,  75.,  76.,  77.,  78.,  79.,  80.,  81.],
       [ 91.,  92.,  93.,  94.,  95.,  96.,  97.,  98.,  99.],
       [109., 110., 111., 112., 113., 114., 115., 116., 117.],
       [127., 128., 129., 130., 131., 132., 133., 134., 135.],
       [145., 146., 147., 148., 149., 150., 151., 152., 153.],
       [163., 164., 165., 166., 167., 168., 169., 170., 171.],
       [181., 182., 183., 184., 185., 186., 187., 188., 189.]])

In [37]: m[:, :2, :]
Out[37]:
array([[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.],
       [19., 20., 21., 22., 23., 24., 25., 26., 27.],
       [37., 38., 39., 40., 41., 42., 43., 44., 45.],
       [55., 56., 57., 58., 59., 60., 61., 62., 63.],
       [73., 74., 75., 76., 77., 78., 79., 80., 81.],
       [91., 92., 93., 94., 95., 96., 97., 98., 99.],
       [109., 110., 111., 112., 113., 114., 115., 116., 117.],
       [127., 128., 129., 130., 131., 132., 133., 134., 135.],
       [145., 146., 147., 148., 149., 150., 151., 152., 153.],
       [163., 164., 165., 166., 167., 168., 169., 170., 171.],
       [181., 182., 183., 184., 185., 186., 187., 188., 189.]])

In [38]: m[:, :-1, :]
Out[38]:
array([[181., 182., 183., 184., 185., 186., 187., 188., 189.],
       [172., 173., 174., 175., 176., 177., 178., 179., 180.],
       [163., 164., 165., 166., 167., 168., 169., 170., 171.],
       [154., 155., 156., 157., 158., 159., 160., 161., 162.],
       [145., 146., 147., 148., 149., 150., 151., 152., 153.],
       [136., 137., 138., 139., 140., 141., 142., 143., 144.],
       [127., 128., 129., 130., 131., 132., 133., 134., 135.],
       [118., 119., 120., 121., 122., 123., 124., 125., 126.],
       [109., 110., 111., 112., 113., 114., 115., 116., 117.],
       [100., 101., 102., 103., 104., 105., 106., 107., 108.],
       [ 91.,  92.,  93.,  94.,  95.,  96.,  97.,  98.,  99.],
       [ 82.,  83.,  84.,  85.,  86.,  87.,  88.,  89.,  90.],
       [ 73.,  74.,  75.,  76.,  77.,  78.,  79.,  80.,  81.],
       [ 64.,  65.,  66.,  67.,  68.,  69.,  70.,  71.,  72.],
       [ 55.,  56.,  57.,  58.,  59.,  60.,  61.,  62.,  63.],
       [ 46.,  47.,  48.,  49.,  50.,  51.,  52.,  53.,  54.],
       [ 37.,  38.,  39.,  40.,  41.,  42.,  43.,  44.,  45.],
       [ 28.,  29.,  30.,  31.,  32.,  33.,  34.,  35.,  36.],
       [ 19.,  20.,  21.,  22.,  23.,  24.,  25.,  26.,  27.],
       [ 10.,  11.,  12.,  13.,  14.,  15.,  16.,  17.,  18.],
       [  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.]])

In [39]: a[np.r_[1:len(a), 0]]
Out[39]:
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.]])

```

```

    [ 7.,  8., 10.],
    [ 1.,  2.,  3.])

In [40]: # --- transpose and arithmetic ---

In [41]: a.transpose()
Out[41]:
array([[ 1.,  4.,  7.],
       [ 2.,  5.,  8.],
       [ 3.,  6., 10.]])

In [42]: a.T
Out[42]:
array([[ 1.,  4.,  7.],
       [ 2.,  5.,  8.],
       [ 3.,  6., 10.]])

In [43]: c = a + 1j * np.eye(3)

In [44]: c.conj().transpose()
Out[44]:
array([[ 1.-1.j,  4.-0.j,  7.-0.j],
       [ 2.-0.j,  5.-1.j,  8.-0.j],
       [ 3.-0.j,  6.-0.j, 10.-1.j]])

In [45]: c.conj().T
Out[45]:
array([[ 1.-1.j,  4.-0.j,  7.-0.j],
       [ 2.-0.j,  5.-1.j,  8.-0.j],
       [ 3.-0.j,  6.-0.j, 10.-1.j]])

In [46]: a @ a
Out[46]:
array([[ 30.,  36.,  45.],
       [ 66.,  81., 102.],
       [109., 134., 169.]])

In [47]: a * a
Out[47]:
array([[ 1.,  4.,  9.],
       [16., 25., 36.],
       [49., 64., 100.]])

In [48]: a / a
Out[48]:
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])

In [49]: a ** 3
Out[49]:
array([[ 1.,  8., 27.],
       [ 64., 125., 216.],
       [343., 512., 1000.]])

In [50]: # --- comparison and boolean selection ---

In [51]: (u > 0.5)
Out[51]:
array([[False,  True, False],
       [ True, False,  True]])

In [52]: np.nonzero(u > 0.5)
Out[52]: (array([0, 1, 1]), array([1, 0, 2]))

In [53]: u[:, np.nonzero(v > 0.5)[0]]
Out[53]:
array([[0.8, 0.5],

```

```

[0.4, 0.9]])

In [54]: u[:, v.T > 0.5]
Out[54]:
array([[0.8, 0.5],
       [0.4, 0.9]])

In [55]: w = u.copy(); w[w < 0.5] = 0; w
Out[55]:
array([[0. , 0.8, 0.5],
       [0.6, 0. , 0.9]])

In [56]: u * (u > 0.5)
Out[56]:
array([[0. , 0.8, 0. ],
       [0.6, 0. , 0.9]])

In [57]: w[:] = 3; w
Out[57]:
array([[3., 3., 3.],
       [3., 3., 3.]])

In [58]: # --- copies vs references ---

In [59]: x = a.copy()

In [60]: y = x.copy(); y
Out[60]:
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.],
       [ 7.,  8., 10.]])

In [61]: y = x[1, :].copy(); y
Out[61]: array([4., 5., 6.])

In [62]: y = x.flatten(); y
Out[62]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8., 10.])

In [63]: x.flatten('F')
Out[63]: array([ 1.,  4.,  7.,  2.,  5.,  8.,  3.,  6., 10.])

In [64]: # --- ranges and constructors ---

In [65]: np.arange(1., 11.)
Out[65]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])

In [66]: np.r_[1.:11.]
Out[66]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])

In [67]: np.r_[1:10:10j]
Out[67]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])

In [68]: np.arange(10.)
Out[68]: array([0., 1., 2., 3., 4., 5., 6., 7., 8., 9.])

In [69]: np.r_[10:]
Out[69]: array([0., 1., 2., 3., 4., 5., 6., 7., 8., 9.])

In [70]: np.r_[9:10j]
Out[70]: array([0., 1., 2., 3., 4., 5., 6., 7., 8., 9.])

In [71]: np.arange(1., 11.)[:, np.newaxis]
Out[71]:
array([[ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.],

```

```

        [ 6.],
        [ 7.],
        [ 8.],
        [ 9.],
        [10.]])

In [72]: np.zeros((3, 4))
Out[72]:
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])

In [73]: np.zeros((3, 4, 5))
Out[73]:
array([[[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]],

       [[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]],

       [[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])

In [74]: np.ones((3, 4))
Out[74]:
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])

In [75]: np.eye(3)
Out[75]:
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])

In [76]: np.diag(a)
Out[76]: array([ 1.,  5., 10.])

In [77]: np.diag(v, 0)
Out[77]:
array([[0.1, 0. , 0. ],
       [0. , 0.6, 0. ],
       [0. , 0. , 0.9]])

In [78]: from numpy.random import default_rng

In [79]: rng = default_rng(42)

In [80]: rng.random((3, 4))
Out[80]:
array([[0.774 , 0.4389, 0.8586, 0.6974],
       [0.0942, 0.9756, 0.7611, 0.7861],
       [0.1281, 0.4504, 0.3708, 0.9268]])

In [81]: np.random.rand(3, 4)
Out[81]:
array([[0.2841, 0.1577, 0.2679, 0.6935],
       [0.6125, 0.216 , 0.6322, 0.553 ],
       [0.2134, 0.6721, 0.1605, 0.9768]])

In [82]: np.linspace(1, 3, 4)
Out[82]: array([1.        , 1.6667, 2.3333, 3.        ])

```

```

In [83]: np.mgrid[0:9., 0:6.]
Out[83]:
array([[0., 0., 0., 0., 0., 0.],
       [1., 1., 1., 1., 1., 1.],
       [2., 2., 2., 2., 2., 2.],
       [3., 3., 3., 3., 3., 3.],
       [4., 4., 4., 4., 4., 4.],
       [5., 5., 5., 5., 5., 5.],
       [6., 6., 6., 6., 6., 6.],
       [7., 7., 7., 7., 7., 7.],
       [8., 8., 8., 8., 8., 8.]],

      [[0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.]])

In [84]: np.meshgrid(np.r_[0:9.], np.r_[0:6.])
Out[84]:
(array([[0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.]]) ,
 array([[0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [2., 2., 2., 2., 2., 2., 2., 2., 2.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [5., 5., 5., 5., 5., 5., 5., 5., 5.]]) )

In [85]: np.ogrid[0:9., 0:6.]
Out[85]:
(array([[0.],
       [1.],
       [2.],
       [3.],
       [4.],
       [5.],
       [6.],
       [7.],
       [8.]]),
 array([[0., 1., 2., 3., 4., 5.]]) )

In [86]: np.ix_(np.r_[0:9.], np.r_[0:6.])
Out[86]:
(array([[0.],
       [1.],
       [2.],
       [3.],
       [4.],
       [5.],
       [6.],
       [7.],
       [8.]]),
 array([[0., 1., 2., 3., 4., 5.]]) )

In [87]: np.meshgrid([1, 2, 4], [2, 4, 5])
Out[87]:
(array([[1, 2, 4],
       [1, 2, 4],
       [1, 2, 4],
       [1, 2, 4],
       [1, 2, 4],
       [1, 2, 4],
       [1, 2, 4],
       [1, 2, 4],
       [1, 2, 4]]),
 array([[2, 4, 5],
       [2, 4, 5],
       [2, 4, 5],
       [2, 4, 5],
       [2, 4, 5],
       [2, 4, 5],
       [2, 4, 5],
       [2, 4, 5],
       [2, 4, 5]]))

```

```

        [1, 2, 4]],
        array([[2, 2, 2],
               [4, 4, 4],
               [5, 5, 5]])

In [88]: np.ix_([1, 2, 4], [2, 4, 5])
Out[88]:
(array([[1],
        [2],
        [4]]),
 array([[2, 4, 5]]))

In [89]: np.tile(p, (2, 3))
Out[89]:
array([[1., 2., 1., 2., 1., 2.],
       [3., 4., 3., 4., 3., 4.],
       [1., 2., 1., 2., 1., 2.],
       [3., 4., 3., 4., 3., 4.]])

In [90]: # --- concatenation ---

In [91]: np.concatenate((p, q), 1)
Out[91]:
array([[1., 2., 5., 6.],
       [3., 4., 7., 8.]])

In [92]: np.hstack((p, q))
Out[92]:
array([[1., 2., 5., 6.],
       [3., 4., 7., 8.]])

In [93]: np.column_stack((p, q))
Out[93]:
array([[1., 2., 5., 6.],
       [3., 4., 7., 8.]])

In [94]: np.c_[p, q]
Out[94]:
array([[1., 2., 5., 6.],
       [3., 4., 7., 8.]])

In [95]: np.concatenate((p, q))
Out[95]:
array([[1., 2.],
       [3., 4.],
       [5., 6.],
       [7., 8.]])

In [96]: np.vstack((p, q))
Out[96]:
array([[1., 2.],
       [3., 4.],
       [5., 6.],
       [7., 8.]])

In [97]: np.r_[p, q]
Out[97]:
array([[1., 2.],
       [3., 4.],
       [5., 6.],
       [7., 8.]])

In [98]: # --- max / norm ---

In [99]: a.max()
Out[99]: np.float64(10.0)

In [100]: np.nanmax(a)

```

```

Out[100]: np.float64(10.0)

In [101]: a.max(0)
Out[101]: array([ 7.,  8., 10.])

In [102]: a.max(1)
Out[102]: array([ 3.,  6., 10.])

In [103]: np.maximum(p, q)
Out[103]:
array([[5., 6.],
       [7., 8.]])

In [104]: np.sqrt(v @ v)
Out[104]: np.float64(1.0862780491200215)

In [105]: np.linalg.norm(v)
Out[105]: np.float64(1.0862780491200215)

In [106]: # --- logical and bitwise ---

In [107]: ba = np.array([True, False, True]); bb = np.array([True, True, False])

In [108]: np.logical_and(ba, bb)
Out[108]: array([ True, False, False])

In [109]: np.logical_or(ba, bb)
Out[109]: array([ True,  True,  True])

In [110]: np.array([12, 10, 7]) & np.array([10, 6, 5])
Out[110]: array([8, 2, 5])

In [111]: np.array([12, 10, 7]) | np.array([10, 6, 5])
Out[111]: array([14, 14,  7])

In [112]: # --- linear algebra ---

In [113]: scipy.linalg.inv(a)
Out[113]:
array([[ -0.6667, -1.3333,  1.    ],
       [ -0.6667,  3.6667, -2.    ],
       [  1.    , -2.    ,  1.    ]])

In [114]: scipy.linalg.pinv(m[:3, :4])
Out[114]:
array([[ -0.3056, -0.1    ,  0.1056],
       [ -0.1111, -0.0333,  0.0444],
       [  0.0833,  0.0333, -0.0167],
       [  0.2778,  0.1    , -0.0778]])

In [115]: np.linalg.matrix_rank(a)
Out[115]: np.int64(3)

In [116]: b = np.array([[6.], [15.], [25.]])

In [117]: scipy.linalg.solve(a, b)
Out[117]:
array([[1.],
       [1.],
       [1.]])

In [118]: scipy.linalg.lstsq(m[:, :3], m[:, 3])
Out[118]:
(array([ -0.6667,  0.3333,  1.3333]),
 array(nan),
 np.int64(2),
 array([848.748 ,  3.3029,  0.    ]))

```

```

In [119]: br = np.array([[6., 15., 25.]])

In [120]: scipy.linalg.solve(a.T, br.T).T
Out[120]: array([[11., -3., 1.]])

In [121]: U, S, Vh = scipy.linalg.svd(a); V = Vh.T

In [122]: U
Out[122]:
array([[ -0.2093,  0.9644,  0.1617],
       [ -0.5038,  0.0353, -0.8631],
       [ -0.838 , -0.2621,  0.4785]])

In [123]: S
Out[123]: array([17.4125,  0.8752,  0.1969])

In [124]: V
Out[124]:
array([[ -0.4647, -0.8333,  0.2995],
       [ -0.5538,  0.0095, -0.8326],
       [ -0.691 ,  0.5528,  0.4659]])

In [125]: spd = np.array([[4., 2.], [2., 3.]])

In [126]: scipy.linalg.cholesky(spd)
Out[126]:
array([[2., 1. ],
       [0., 1.4142]])

In [127]: D, V = scipy.linalg.eig(a)

In [128]: D
Out[128]: array([16.7075+0.j, -0.9057+0.j,  0.1982+0.j])

In [129]: V
Out[129]:
array([[ -0.2235, -0.8658,  0.2783],
       [ -0.5039,  0.0857, -0.8318],
       [ -0.8343,  0.4929,  0.4802]])

In [130]: spd2 = np.array([[2., 0.], [0., 1.]])

In [131]: D, V = scipy.linalg.eig(spd, spd2)

In [132]: D
Out[132]: array([1.+0.j, 4.+0.j])

In [133]: V
Out[133]:
array([[ 0.7071,  0.4472],
       [-0.7071,  0.8944]])

In [134]: big = np.diag(np.arange(1., 7.))

In [135]: D, V = eigs(big, k=3)

In [136]: D
Out[136]: array([6.+0.j, 5.+0.j, 4.+0.j])

In [137]: V
Out[137]:
array([[ -0.+0.j,  0.+0.j, -0.+0.j],
       [ -0.+0.j, -0.+0.j, -0.+0.j],
       [ -0.+0.j, -0.+0.j, -0.+0.j],
       [ 0.+0.j, -0.+0.j,  1.+0.j],
       [ 0.+0.j,  1.+0.j,  0.+0.j],
       [ 1.+0.j, -0.+0.j, -0.+0.j]])

```

```

In [138]: Q, R = scipy.linalg.qr(a)

In [139]: Q
Out[139]:
array([[ -0.1231,  0.9045,  0.4082],
       [ -0.4924,  0.3015, -0.8165],
       [ -0.8616, -0.3015,  0.4082]])

In [140]: R
Out[140]:
array([[ -8.124 , -9.6011, -11.9399],
       [  0.    ,  0.9045,  1.5076],
       [  0.    ,  0.    ,  0.4082]])

In [141]: P, L, U = scipy.linalg.lu(a)

In [142]: P
Out[142]:
array([[0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.]])

In [143]: L
Out[143]:
array([[1.    , 0.    , 0.    ],
       [0.1429, 1.    , 0.    ],
       [0.5714, 0.5    , 1.    ]])

In [144]: U
Out[144]:
array([[ 7.    ,  8.    , 10.    ],
       [ 0.    , 0.8571,  1.5714],
       [ 0.    ,  0.    , -0.5    ]])

In [145]: np.allclose(a, P @ L @ U)
Out[145]: True

In [146]: cg(spd, np.array([1., 2.]))
Out[146]: (array([-0.125,  0.75 ]), 0)

In [147]: np.fft.fft(v)
Out[147]: array([ 1.6+0.j      , -0.65+0.2598j, -0.65-0.2598j])

In [148]: np.fft.ifft(np.fft.fft(v))
Out[148]: array([0.1+0.j, 0.6+0.j, 0.9+0.j])

In [149]: # --- sorting ---

In [150]: np.sort(a)
Out[150]:
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.],
       [ 7.,  8., 10.]])

In [151]: as0 = a.copy(); as0.sort(axis=0); as0
Out[151]:
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.],
       [ 7.,  8., 10.]])

In [152]: np.sort(a, axis=1)
Out[152]:
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.],
       [ 7.,  8., 10.]])

In [153]: as1 = a.copy(); as1.sort(axis=1); as1
Out[153]:

```

```

array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.],
       [ 7.,  8., 10.]])

In [154]: s = np.array([[3., 9.], [1., 8.], [2., 7.]])

In [155]: I = np.argsort(s[:, 0]); bs = s[I, :]; bs
Out[155]:
array([[1., 8.],
       [2., 7.],
       [3., 9.]])

In [156]: Z = np.c_[np.ones(5), np.arange(5.)]; yv = np.array([1., 3., 5., 7., 9.])

In [157]: scipy.linalg.lstsq(Z, yv)
Out[157]: (array([1., 2.]), array(0.), np.int64(2), array([5.7886, 1.2216]))

In [158]: sig = np.sin(np.linspace(0, 2 * np.pi, 20)); qd = 4

In [159]: signal.resample(sig, int(np.ceil(len(sig) / qd)))
Out[159]: array([ 0.129 ,  0.9349,  0.4995, -0.7589, -0.8045])

In [160]: np.unique(np.array([3, 1, 2, 3, 1]))
Out[160]: array([1, 2, 3])

In [161]: np.zeros((1, 3, 1)).squeeze()
Out[161]: array([0., 0., 0.])

```

Task 3

I ran the script on a server without a display, so I added a `savefig` line to save the figure.

```

In [1]: import matplotlib.pyplot as plt

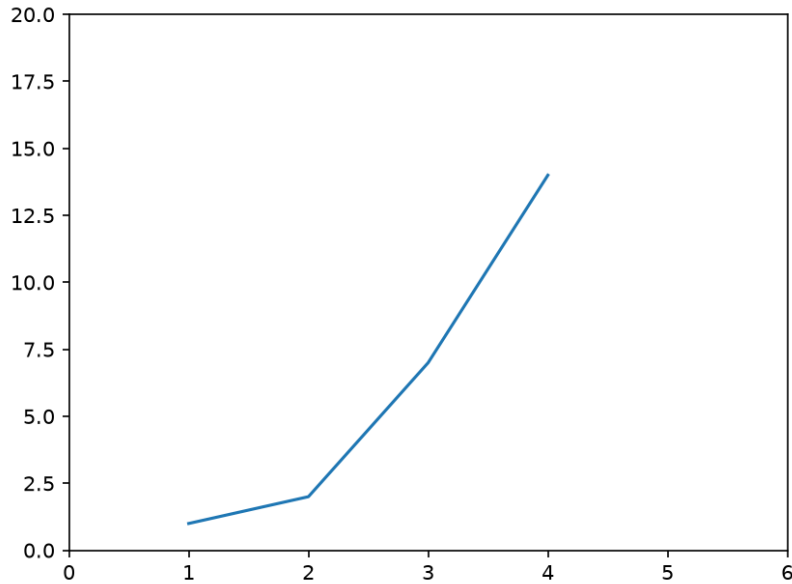
In [2]: plt.plot([1,2,3,4], [1,2,7,14])
Out[2]: [<matplotlib.lines.Line2D at 0x7f1d97a147d0>]

In [3]: plt.axis([0, 6, 0, 20])
Out[3]: (np.float64(0.0), np.float64(6.0), np.float64(0.0), np.float64(20.0))

In [4]: plt.show()

In [5]: plt.savefig('task3_figure.png', dpi=150, bbox_inches='tight')

```



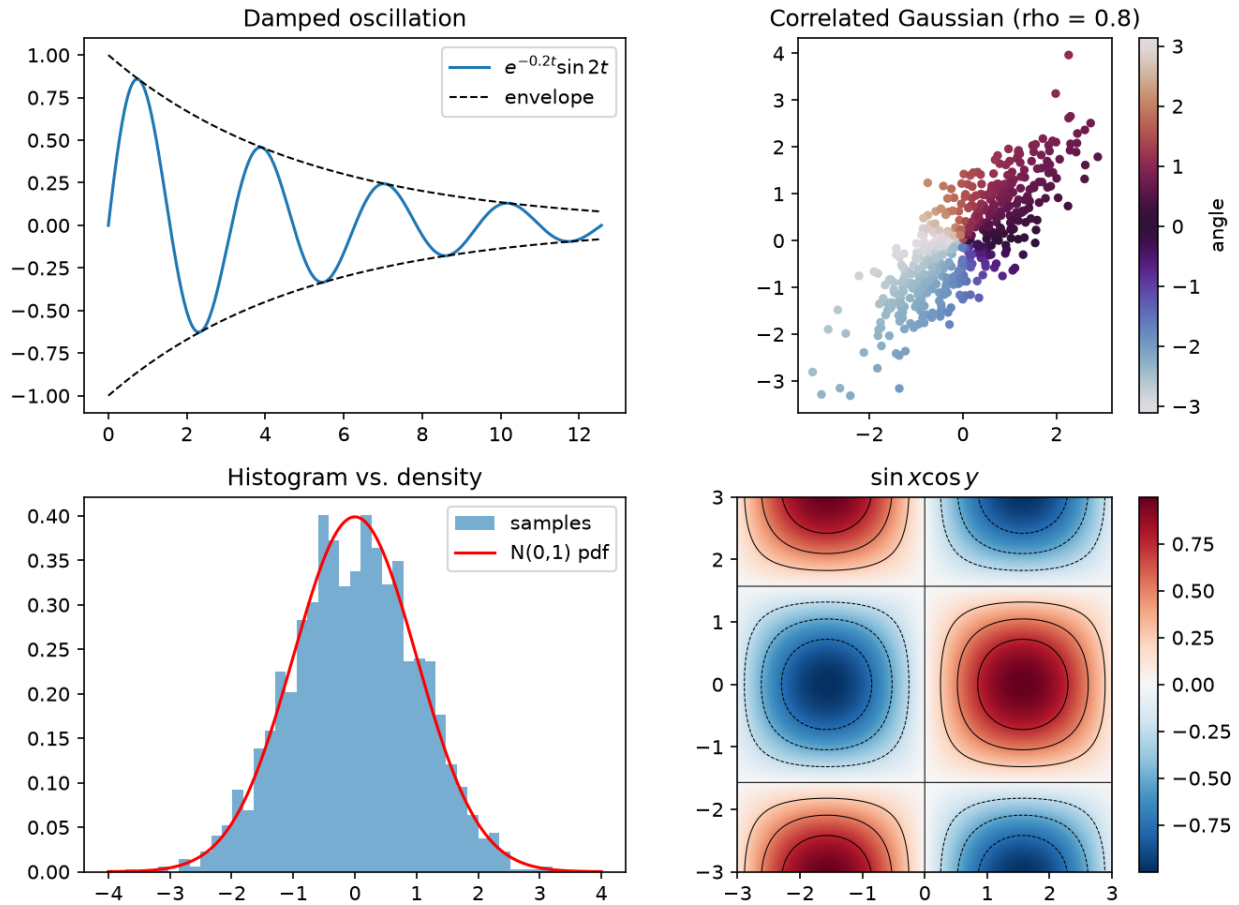
Task 4

Code:

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(576)
fig, ax = plt.subplots(2, 2, figsize=(9, 7))
t = np.linspace(0, 4 * np.pi, 400)
ax[0, 0].plot(t, np.exp(-0.2 * t) * np.sin(2 * t), label=r'$e^{-0.2t}\sin 2t$')
ax[0, 0].plot(t, np.exp(-0.2 * t), 'k--', lw=1, label='envelope')
ax[0, 0].plot(t, -np.exp(-0.2 * t), 'k--', lw=1)
ax[0, 0].set_title('Damped oscillation')
ax[0, 0].legend()
pts = rng.multivariate_normal([0, 0], [[1, 0.8], [0.8, 1]], 500)
sc = ax[0, 1].scatter(pts[:, 0], pts[:, 1], c=np.arctan2(pts[:, 1], pts[:, 0]), cmap='twilight', s=12)
ax[0, 1].set_title('Correlated Gaussian (rho = 0.8)')
ax[0, 1].set_aspect('equal')
fig.colorbar(sc, ax=ax[0, 1], label='angle')
z = rng.standard_normal(2000)
ax[1, 0].hist(z, bins=40, density=True, alpha=0.6, label='samples')
xs = np.linspace(-4, 4, 200)
ax[1, 0].plot(xs, np.exp(-xs**2 / 2) / np.sqrt(2 * np.pi), 'r', label='N(0,1) pdf')
ax[1, 0].set_title('Histogram vs. density')
ax[1, 0].legend()
X, Y = np.meshgrid(np.linspace(-3, 3, 200), np.linspace(-3, 3, 200))
Zg = np.sin(X) * np.cos(Y)
im = ax[1, 1].imshow(Zg, extent=[-3, 3, -3, 3], origin='lower', cmap='RdBu_r')
ax[1, 1].contour(X, Y, Zg, levels=8, colors='k', linewidths=0.5)
ax[1, 1].set_title(r'$\sin x \cos y$')
fig.colorbar(im, ax=ax[1, 1])
fig.suptitle('ELEC 576 Assignment 0, Task 4: a Matplotlib figure of my choice')
```

```
fig.tight_layout()
plt.show()
fig.savefig('task4_figure.png', dpi=150, bbox_inches='tight')
```

ELEC 576 Assignment 0, Task 4: a Matplotlib figure of my choice



Task 5

GitHub account: [AnqiLi24](#)

Task 6

Project (VS Code), pushed to GitHub: <https://github.com/AnqiLi24/elec576-assignment0>